

Resumen - Inteligencia de Datos y RAG Avanzado

Esta formación se centra en **construir soluciones de IA conectadas a datos reales** (documentos, PDFs, páginas web y datasets tipo CSV) mediante **RAG (Retrieval-Augmented Generation)** y técnicas avanzadas de recuperación. El objetivo es pasar de “un chatbot que responde” a **una aplicación inteligente** que:

- encuentra información relevante en tus fuentes,
 - la organiza como contexto,
 - responde con precisión (y idealmente con trazabilidad),
 - y puede automatizar tareas alrededor de esos datos (análisis, reportes, visualización, etc.).
-

1) Idea central: ¿por qué RAG?

Los modelos de lenguaje (LLMs) son excelentes redactando y razonando, pero:

- pueden **alucinar** (inventar),
- no conocen tus documentos privados,
- y pueden estar desactualizados.

RAG resuelve esto al agregar un paso de **búsqueda/recuperación** antes de generar la respuesta. En vez de pedirle al LLM “que se acuerde”, le pedís que:

1. **busque** en tus fuentes,
2. **seleccione** lo relevante,
3. y **responda** usando ese material.

Resultado práctico: respuestas más confiables, orientadas a tus datos, con mejor control.

2) Qué vas a construir (arquitectura típica)

En esta ruta, el “producto” final suele ser un pipeline con estos componentes:

a) Ingesta de datos (documentos → texto)

- Extracción de texto desde PDFs, páginas web, notas, archivos, etc.
- Normalización: limpieza básica, detección de secciones, metadatos (fuente, fecha, tema).

b) Chunking (fragmentación)

Dividir el contenido en “trozos” (chunks) para:

- recuperar partes específicas,
- evitar pasar documentos enteros al LLM,
- y mantener contexto útil dentro de la ventana del modelo.

Decisiones importantes:

- tamaño del chunk,
- solapamiento,
- segmentación por títulos/estructura,
- preservación de metadatos por chunk.

c) Embeddings (texto → vectores)

Transformar cada chunk en un vector numérico que capture similitud semántica.

- Te permite buscar por “significado”, no solo por palabras exactas.

d) Base vectorial / índice

Guardar embeddings + metadatos en un motor de búsqueda vectorial (vector store).

- Se habilita búsqueda por similitud + filtros por metadatos.

e) Retrieval (recuperación)

Dada una pregunta:

- convertirla en embedding,
- recuperar los chunks más similares,
- y (opcional) reordenarlos con técnicas de re-ranking.

f) Generación (LLM)

El LLM recibe:

- la pregunta,
 - el contexto recuperado,
 - y reglas de respuesta (prompt): formato, tono, límites, referencias.
-

3) Contenidos clave en profundidad

3.1 RAG + agentes (cuando “buscar y responder” no alcanza)

Un sistema RAG básico responde una pregunta puntual. Un agente agrega:

- planificación (qué pasos seguir),
- decisiones (qué herramienta usar),
- iteración (si la primera búsqueda no alcanza, reformular y volver a buscar),
- acciones (generar reporte, extraer tabla, comparar fuentes, etc.).

Ejemplos de tareas de agente con RAG:

- “Leé este PDF, extraé los 10 puntos clave y generá un resumen ejecutivo.”
- “Buscá inconsistencias entre dos documentos y listalas.”
- “Respondé con cita de fuente y sección.”
- “Si falta información, pedime aclaraciones.”

3.2 Técnicas avanzadas: chunking, vectorización y calidad

Esta parte suele marcar la diferencia entre un demo y un sistema útil.

Chunking y trade-offs:

- Chunks muy chicos: recuperás exactitud pero perdés contexto.
- Chunks muy grandes: tenés contexto pero la recuperación se vuelve ruidosa.
- Segmentación inteligente: por títulos/headers, por párrafos, por longitud y estructura.

Embeddings:

- Elección del modelo de embeddings impacta la calidad.
- Buen uso de metadatos: fuente, tema, fecha, tipo de documento, autor.

Mejoras típicas:

- re-ranking (para mejorar el top-k),
- multi-query / query expansion (generar variantes de la pregunta),
- self-querying (convertir preguntas en filtros por metadatos),
- compresión del contexto (resumir chunks antes de pasarlos al LLM).

3.3 Evaluación y monitoreo (pasar a algo confiable)

Una vez armado el pipeline, se trabaja:

- medición de recuperación (¿estás trayendo el chunk correcto?),
- medición de respuesta (¿usa el contexto? ¿se inventa cosas?),
- tracing (ver qué se recuperó y por qué),
- datasets de evaluación (preguntas “golden” para comparar versiones).

Herramientas como **LangSmith** (u otras) ayudan a:

- trazar ejecuciones,
 - comparar prompts,
 - detectar fallos comunes,
 - mejorar iterativamente.
-

4) Automatizando análisis de datos con agentes (CSV / dataframes)

Esta ruta también conecta RAG y agentes con **análisis de datos**.

Qué tipo de asistente se busca

Un asistente que:

- entiende un dataset (CSV),
- responde preguntas (“¿qué provincias tienen más entregas?”),
- genera reportes (exploratorio / estadístico),
- y crea visualizaciones.

Herramientas típicas del agente

- **Reporte exploratorio:** nulos, duplicados, strings “nan”, tipos, cardinalidad, outliers.
- **Resumen estadístico:** describe, percentiles, distribución, correlaciones simples.
- **Visualización:** gráficos con seaborn/matplotlib, reglas estéticas, títulos, ejes, etiquetas.
- **Ejecución de código:** un entorno controlado (por ejemplo, exec con globals limitados).

Orquestación

El agente decide:

- si primero necesita explorar el dataset,
 - si necesita generar un gráfico,
 - o si puede contestar con estadística descriptiva.
-

5) Entregables / resultado final esperado

Al terminar, deberías poder construir al menos:

1. **Un RAG funcional** (ingesta → embeddings → vector store → retrieval → respuesta).
2. **Un agente** que itera y usa herramientas (buscar, reformular, comparar, resumir).
3. **Un mini-producto** con interfaz (por ejemplo Streamlit) que permita usarlo sin tocar código.

Lo más importante: que tu solución pase de “parece que anda” a “puedo confiar en esto para casos reales”, gracias a evaluación, trazas y mejoras.

Glosario rápido

- **RAG:** técnica para responder usando contexto recuperado de fuentes externas.
- **Chunking:** dividir documentos en fragmentos recuperables.
- **Embeddings:** representación vectorial semántica del texto.
- **Vector store:** base/índice para búsqueda por similitud.
- **Retriever:** componente que trae los chunks más relevantes.
- **Re-ranking:** reordenar resultados para priorizar lo más útil.
- **Agente:** sistema que planifica, decide, usa herramientas e itera.